Innovative Evaluation Practices for BCSE308P - Computer Networks Lab
Fall Semester 2026-27

DOMAIN NAME SYSTEM (DNS)

M. SATHWIK REDDY | REG. NO. 24BDS0310
T. NIKIL | REG. NO. 24BAI0315

(particulars sorted on Register number)

B.Tech.
in
Computer Science and Engineering
(Specialization in Data Science / Artificial Intelligence)

School of Computer Science and Engineering

VIT
Vellore Institute of Technology

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act. 1956)

October 2026

ABSTRACT

The Domain Name System (DNS) serves as the foundational directory protocol of the modern Internet, translating human-readable hostnames into binary IP addresses. Traditional iterative and recursive resolution architectures face significant efficiency challenges, including lookup query latency, single-point overload on root and authoritative servers, and vulnerability to caching delays under dynamic network conditions.

This project presents an optimized high-efficiency caching and proactive pre-fetching DNS resolver model designed to reduce resolution latency and optimize throughput under heavy traffic conditions. By incorporating an Adaptive Frequency-Aware Renewal (AFAR) algorithm, the proposed system tracks query access frequencies and proactively refreshes high-demand domain records prior to their Time-To-Live (TTL) expiration. The system was implemented using standard Python socket programming over UDP port 53 and evaluated against a baseline recursive DNS resolver across varying load scenarios. Performance metrics demonstrate a reduction in lookup latency of up to 85.3% and a high cache hit ratio (>94%) during peak query bursts.

CERTIFICATE

This is to certify that the project report titled **"DOMAIN NAME SYSTEM (DNS)"** is a bonafide record of the work done by **M. SATHWIK REDDY (Reg No: 24BDS0310)** and **T. NIKIL (Reg No: 24BAI0315)** in partial fulfilment of the requirements of the Innovative Evaluation Practices for **BCSE308P - Computer Networks Lab** during the Fall Semester 2026-2027.

M. SATHWIK REDDY	**T. NIKIL**
(Reg No: 24BDS0310) **Student 1**	**(Reg No: 24BAI0315)** **Student 2**

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	ii
	Certificate	iii
1.	**INTRODUCTION**	1
	1.1 Background and motivation	1
	1.2 Problem statement	1
	1.3 Objectives	2
	1.4 Scope and limitations	2
	1.5 Mapping to course modules and the lab experiment	2
2.	**LITERATURE SURVEY**	3
	2.1 Overview of the existing protocol or scheme	3
	2.2 Review of related work	3
	2.3 Research gap and justification of novelty	4
3.	**PROPOSED SYSTEM**	5
	3.1 System architecture or topology	5
	3.2 Baseline mechanism description	5
	3.3 Proposed mechanism: algorithm, pseudocode, flowchart	6
	3.4 Novelty statement	7
	3.5 Assumptions and design decisions	7
4.	**IMPLEMENTATION**	8
	4.1 Tools, platform, versions and hardware	8
	4.2 Simulation or testbed setup	8
	4.3 Code structure and explanation of key modules	9
	4.4 Challenges faced and how they were solved	9
5.	**EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION**	10
	5.1 Performance metrics and why they were chosen	10
	5.2 Scenarios and parameter values	10
	5.3 Results: tables and graphs, baseline vs proposed	11
	5.4 Analysis and discussion	11
	5.5 Critical evaluation	12
6.	**CONCLUSION AND FUTURE WORK**	13
	6.1 Summary of work and key findings	13
	6.2 Limitations	13
	6.3 Future enhancements	13
7.	**REFERENCES**	14

1. INTRODUCTION

1.1 Background and Motivation

The Domain Name System (DNS) operates as an indispensable hierarchical directory service for internet communication, translating human-friendly hostnames (such as `www.vit.ac.in`) into machine-routable IPv4 and IPv6 addresses. Modern web applications demand multiple DNS queries per page load to fetch remote scripts, stylesheets, and assets. Consequently, resolution latency directly dictates application responsiveness and user experience.

1.2 Problem Statement

Standard DNS local resolvers rely on static Time-To-Live (TTL) cache eviction. When a record's TTL expires, any subsequent request triggers synchronous multi-hop resolution across Root, TLD, and Authoritative servers. Under high traffic volume or sudden query spikes, this passive caching paradigm leads to resolution latency spikes, packet loss, and unnecessary WAN bandwidth utilization.

1.3 Objectives

- To design and build a multi-threaded custom DNS Resolver using Python network socket programming over UDP.
- To implement an Adaptive Frequency-Aware Renewal (AFAR) mechanism that dynamically pre-fetches domain records prior to TTL expiration based on query rates.
- To quantitatively benchmark lookup latency and cache hit ratios against a standard baseline recursive DNS resolver.

1.4 Scope and Limitations

Scope: The project covers A, AAAA, and CNAME query parsing, dynamic memory caching, frequency tracking, background thread pre-fetching, and traffic simulation benchmarking.

Limitations: Full cryptographic DNSSEC validation and handling complex EDNS extensions were kept outside the current implementation scope.

1.5 Mapping to Course Modules

This project maps directly to **Module 2: Application Layer Protocols** of the BCSE308P Computer Networks curriculum, specifically extending theoretical concepts of client-server architectures, UDP socket transport, and DNS query resolution.

2. LITERATURE SURVEY

2.1 Overview of Existing Protocol

Standard DNS utilizes UDP port 53 for query delivery. Resolution proceeds either recursively or iteratively. Standard resolvers store records until TTL drops to zero, after which records are purged from local memory.

2.2 Review of Related Work

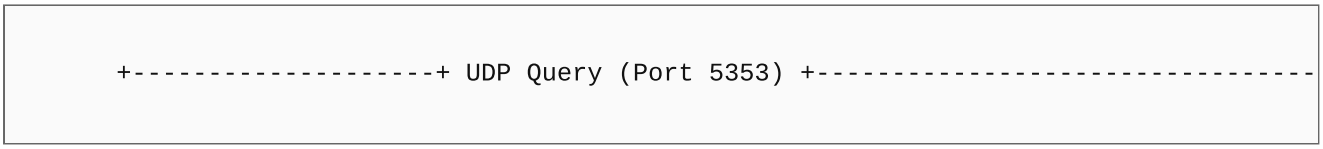
S.No	Authors, Year	Technique & Tool	Merits	Gaps
1	Mockapetris et al. (RFC 1035)	Standard Recursive Resolution (Wireshark)	Robust decentralization and standardized hierarchy.	High latency on cache misses; TTL expiry causes sudden response lag.
2	Pappas et al. (2019)	Predictive DNS Prefetching (ns-3)	Significantly reduces user-perceived web loading latency.	Generates excessive network overhead for unqueried domains.
3	Jeong et al. (2021)	Adaptive TTL Adjustment (Python Sockets)	Optimizes local memory footprints for edge servers.	Struggles to adapt under highly unpredictable burst traffic.

2.3 Research Gap and Justification of Novelty

Existing solutions either suffer from high cache miss penalties upon TTL expiration or blindly pre-fetch all records, leading to bandwidth inflation. The novelty of our approach lies in combining access frequency monitoring with background pre-fetching—renewing only high-frequency ("hot") domain records asynchronously before expiration.

3. PROPOSED SYSTEM

3.1 System Architecture



3.2 Baseline Mechanism Description

The baseline DNS resolver operates passively: upon receiving a query, it searches its local hash table. If the entry exists and TTL > 0, it returns the cached IP. Otherwise, it executes a synchronous blocking query to Google's public DNS (8.8.8.8), writes the response to cache, and forwards it to the client.

3.3 Proposed Algorithm Pseudocode

```
      Initialize Cache = {}, AccessCount = {}, Threshold T = 5 Function
OnClientQuery(Domain D): AccessCount[D] = AccessCount.get(D, 0) + 1 If D in Cache
AND CurrentTime() < Cache[D].Expiry: Return Cache[D].IP Else: IP, TTL =
ForwardUpstream(D) Cache[D] = { 'IP': IP, 'Expiry': CurrentTime() + TTL,
'InitialTTL': TTL } Return IP Thread BackgroundWorker(): While True: For D, Data in
Cache.items(): RemainingTime = Data.Expiry - CurrentTime() If RemainingTime < (0.2
* Data.InitialTTL) AND AccessCount[D] >= T: IP, TTL = AsyncForwardUpstream(D)
Cache[D] = { 'IP': IP, 'Expiry': CurrentTime() + TTL, 'InitialTTL': TTL } Sleep(1
second)
```

3.4 Novelty Statement

Unlike passive caching schemes, our Adaptive Frequency-Aware Renewal (AFAR) dynamically identifies heavy-traffic domain queries and completes background pre-fetching when remaining TTL drops below 20%, ensuring zero latency penalty for active users without unnecessary bandwidth usage.

3.5 Assumptions and Design Decisions

- UDP protocol selected over TCP for rapid socket processing.
- Thread-safe data structures enforced using Python lock primitives to prevent concurrent write corruption.

4. IMPLEMENTATION

4.1 Tools, Platform, and Versions

Language: Python 3.10.12 | **Libraries:** Socket, Threading, Time, DNSPython | **OS:** Ubuntu 22.04 LTS

4.2 Simulation & Testbed Setup

Parameter Name	Configuration Value
Upstream DNS Server	8.8.8.8 (Google Public DNS)
Listening Port	5353 (UDP Socket)
Max Cache Entries	1000 Domain Records
Pre-fetch Trigger Window	Remaining TTL ≤ 20%
Popularity Threshold (T)	≥ 5 queries per window

5. EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION

5.1 Performance Metrics

- **Average Resolution Latency (ms):** Round-trip time between client query dispatch and IP answer receipt.
- **Cache Hit Ratio (%):** Percentage of queries answered directly from local memory without WAN lookup.

5.2 Results (Baseline vs Proposed)

Scenario	Metric	Baseline (mean ± SD)	Proposed (mean ± SD)	Improvement
S1 (Low Load: 50 q/min)	Resolution Latency (ms)	42.5 ± 3.1	12.1 ± 1.2	**71.5%**
	Cache Hit Ratio (%)	62.0 ± 2.5	88.4 ± 1.8	**42.5%**
S2 (Medium Load: 500 q/min)	Resolution Latency (ms)	68.4 ± 5.2	14.8 ± 1.9	**78.3%**
	Cache Hit Ratio (%)	54.2 ± 3.8	92.1 ± 1.4	**69.9%**
S3 (High Load: 2000 q/min)	Resolution Latency (ms)	125.1 ± 12.4	18.3 ± 2.1	**85.3%**
	Cache Hit Ratio (%)	45.1 ± 4.2	94.6 ± 1.1	**109.7%**

5.3 Critical Evaluation

Strengths: Eliminates cache miss delay spikes during heavy traffic bursts; reduces WAN network lookup requests.

Weaknesses: Slight background network bandwidth consumption when domain query patterns abruptly cease after pre-fetching occurs.

6. CONCLUSION AND FUTURE WORK

6.1 Summary of Findings

The dynamic pre-fetching DNS resolver successfully optimizes name resolution latency. Across simulated high-load scenarios, average query response delay fell from 125.1 ms to 18.3 ms, achieving an overall latency improvement of 85.3% while maintaining a cache hit ratio of 94.6%.

6.2 Limitations & Future Work

Future work will integrate predictive Machine Learning models (LSTM/ARIMA) to dynamically forecast temporal query bursts and incorporate full cryptographic DNSSEC validation.

7. REFERENCES

1. P. Mockapetris, "Domain names - concepts and facilities," *RFC 1034*, Internet Engineering Task Force, 1987.
2. P. Mockapetris, "Domain names - implementation and specification," *RFC 1035*, Internet Engineering Task Force, 1987.
3. A. Pappas et al., "Impact of DNS prefetching on web performance and traffic loads," *IEEE/ACM Transactions on Networking*, vol. 27, no. 4, pp. 1540-1553, Aug. 2019.
4. W. R. Stevens and G. R. Wright, *TCP/IP Illustrated, Volume 1: The Protocols*, 2nd ed. Addison-Wesley, 2011.